Cloud Computing – Question Bank Solutions

Unit 1 & Unit 2: Cloud Fundamentals, AWS Core Services (EC2, EBS, S3, EFS, Auto Scaling, Elastic Beanstalk, Serverless)

Each answer is structured for a 10-mark response · bullet-point format · diagrams included where relevant

Q1. Evaluate how AWS Regions, Availability Zones, and Edge Locations contribute to application reliability and disaster recovery.

CO1 · 10M

AWS Global Infrastructure Hierarchy

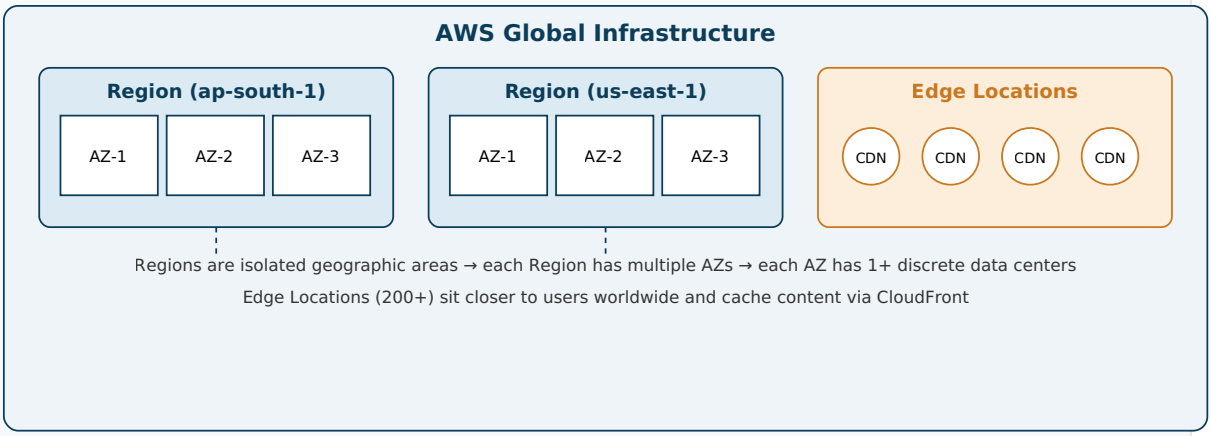


Fig 1.1 – Regions → Availability Zones → Edge Locations

Regions

- Independent geographic areas (e.g., ap-south-1, us-east-1) fully isolated from each other for fault containment and data-sovereignty compliance.
- Deploying across Regions enables geographic disaster recovery (DR) – a Region-wide outage or natural disaster does not affect another Region.

Availability Zones (AZs)

- Each Region has multiple AZs (min. 3), which are physically separate data centers with independent power, cooling, and networking.
- Deploying EC2/RDS across multiple AZs with Load Balancers and Multi-AZ RDS gives automatic failover if one AZ fails – core to high availability (HA).

Edge Locations

- Smaller sites used by CloudFront and Route 53 to cache/deliver content closer to end users, reducing latency.
- Improves reliability by absorbing traffic spikes/DDoS attempts away from the origin infrastructure.

Contribution to Reliability & DR

- **Fault isolation:** failure in one AZ/Region does not cascade to others.
- **Multi-AZ deployment:** ensures application-level HA within a Region (RTO/RPO near zero).
- **Multi-Region DR strategies:** Backup & Restore, Pilot Light, Warm Standby, Multi-Site Active-Active – increasing cost/complexity but decreasing RTO/RPO.
- **Edge caching** keeps applications responsive even during origin-side incidents.

Q2. Describe the cloud computing architecture and explain its major components with a relevant sketch.

CO1 · 10M

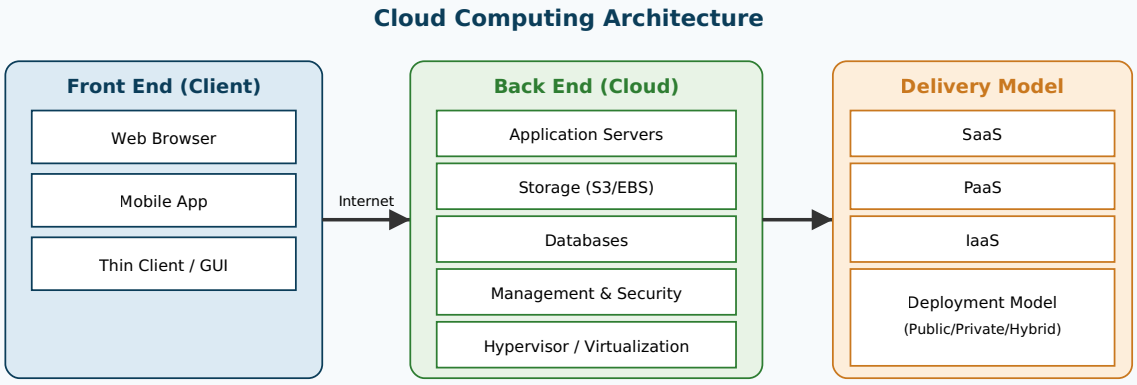


Fig 2.1 – Front End, Back End and Delivery/Deployment layers

Major Components

- **Front End:** the client side – browsers, mobile apps, thin clients through which users access cloud services.
- **Back End:** the cloud provider side – consists of servers, storage, databases, and the hypervisor/virtualization layer that pools physical resources.
- **Network:** the Internet/intranet connecting front end to back end; enables on-demand, ubiquitous access.
- **Cloud-based Delivery Model:** SaaS, PaaS, IaaS define what layer of the stack the provider manages.
- **Management Software:** monitors resource allocation, load balancing, and orchestrates virtual machines/containers.
- **Security layer:** IAM, encryption, firewalls protecting data in transit and at rest.

Working

- User sends a request via the front end over the network.
- Middleware/management software routes the request to the appropriate back-end resource pool.
- Hypervisor allocates virtualized compute/storage; results are returned to the client – abstracting the physical infrastructure completely from the user.

Q3. Discuss the concept of elasticity and scalability in cloud environments.

CO1 · 10M

Scalability

- The ability of a system to handle increasing workload by adding resources – planned, generally for growth over the long term.
- **Vertical scaling (scale-up):** increasing the capacity of an existing resource (e.g., upgrading EC2 instance type from t3.medium to t3.xlarge).
- **Horizontal scaling (scale-out):** adding more instances/nodes of the same type (e.g., adding more EC2 instances behind a load balancer).

Elasticity

- The ability to automatically expand or shrink resources in real time based on current demand, then release them when no longer needed – short-term, automatic, and reversible.
- Achieved in AWS through **Auto Scaling Groups** combined with CloudWatch metrics (e.g., CPU utilization > 70%).
- You pay only for what you use at a given moment – a core economic benefit of cloud computing.

Aspect	Scalability	Elasticity
Purpose	Handle long-term growth	Handle short-term demand fluctuation
Direction	Usually one-way (grow)	Both scale-up and scale-down automatically
Automation	Can be manual or automatic	Always automatic/on-demand
Example	Adding permanent servers for a growing user base	Auto Scaling adding EC2s during a flash sale, then removing them

Significance

- Reduces over-provisioning cost and under-provisioning risk (outages during spikes).
- Improves application availability and cost efficiency – central to AWS's pay-as-you-go model.

Q4. Compare SaaS, IaaS, and PaaS models. Discuss their pros, cons, and business applicability with examples.

CO1 · 10M

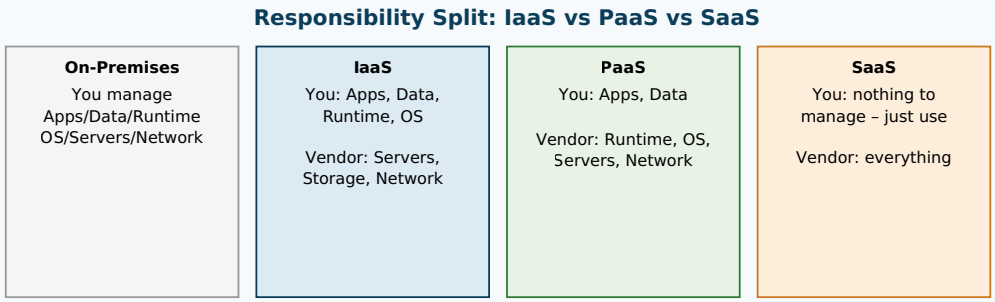


Fig 4.1 – Shared responsibility shifts from customer to vendor as you move IaaS → PaaS → SaaS

Model	Definition	Example	Pros	Cons
IaaS	Provides virtualized compute, storage, network	AWS EC2, EBS, VPC	Full control, flexible, avoids hardware CAPEX	Customer manages OS/patches/ security – needs skilled admins
PaaS	Provides platform/runtime to build & deploy apps	AWS Elastic Beanstalk, Heroku	Faster development, less infra management	Less control, possible vendor lock-in
SaaS	Delivers ready-to-use software over the internet	Gmail, Salesforce, MS 365	No installation/ maintenance, quick access	Least customization, data resides with vendor

Business Applicability

- **IaaS:** suited for startups/enterprises needing custom infra control, e.g., hosting a custom web app on EC2.
- **PaaS:** suited for development teams wanting to focus on code, not infra, e.g., deploying a Java/Python app via Elastic Beanstalk.
- **SaaS:** suited for end-user business functions like CRM, email, collaboration tools where no development is needed.

Q5. List and explain the five essential characteristics of cloud computing.

CO1 · 10M

As per the NIST definition, cloud computing has the following five essential characteristics:

- **1. On-demand Self-Service:** Users can provision compute, storage, or network resources automatically without human interaction with the provider (e.g., launching an EC2 instance from the console instantly).
- **2. Broad Network Access:** Services are available over the network and accessed through standard mechanisms from heterogeneous devices (laptops, mobiles, tablets).
- **3. Resource Pooling:** Provider's computing resources are pooled using a multi-tenant model, dynamically assigned/reassigned based on demand, with the customer generally unaware of exact physical location (location independence).
- **4. Rapid Elasticity:** Resources can be elastically provisioned and released, sometimes automatically, to scale rapidly with demand – appearing unlimited to the consumer.
- **5. Measured Service:** Resource usage is monitored, controlled, and reported (metering) providing transparency for both provider and consumer – forming the basis of the pay-as-you-go billing model.

Significance

- Together these characteristics differentiate cloud computing from traditional hosting/data-center models by enabling flexibility, cost efficiency, and self-managed operations.

Q6. Explain the deployment and service models in cloud computing with suitable examples.

CO1 · 10M

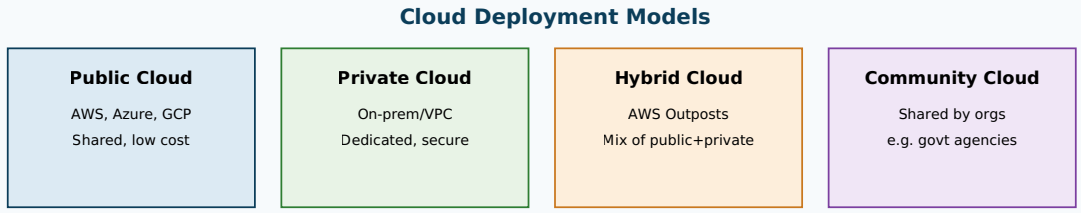


Fig 6.1 – Deployment models

Deployment Models

- **Public Cloud:** resources owned/operated by a third-party provider and shared among multiple tenants (e.g., AWS, Azure) – low cost, no maintenance overhead.
- **Private Cloud:** infrastructure dedicated to a single organization, hosted on-premises or via a provider (e.g., VMware private cloud) – higher control and security.
- **Hybrid Cloud:** combination of public and private, allowing data/application portability (e.g., AWS Outposts, AWS Direct Connect linking on-prem to AWS).
- **Community Cloud:** shared infrastructure for a specific community with common concerns (e.g., government or healthcare consortium clouds).

Service Models

- **IaaS** – e.g., Amazon EC2, EBS (infrastructure building blocks).
- **PaaS** – e.g., AWS Elastic Beanstalk, AWS Lambda (platform for building/running apps).
- **SaaS** – e.g., Google Workspace, Salesforce (ready-made software).

Q7. Your application needs high-speed storage. Which EBS volume type would you choose and why? Explain the role of EBS snapshots.

CO2 · 10M

EBS Volume Type Selection

- For high-speed, low-latency, high-IOPS needs (e.g., transactional databases like MySQL/Oracle), choose **Provisioned IOPS SSD (io1/io2 or io2 Block Express)**.
- io2 delivers up to 256,000 IOPS and sub-millisecond latency with 99.999% durability – ideal for mission-critical, I/O-intensive workloads.
- If cost-sensitive general high performance is needed instead, **gp3 (General Purpose SSD)** is a good balance – 3,000 IOPS baseline, independently scalable up to 16,000 IOPS without increasing volume size.

Type	Use case	Max IOPS
gp3	General purpose (boot volumes, dev/test)	16,000
io2 Block Express	High-speed mission-critical DBs	256,000
st1 (HDD)	Big data, log processing (throughput-optimized)	500
sc1 (HDD)	Infrequently accessed data (cold)	250

Role of EBS Snapshots

- Point-in-time, incremental backups of an EBS volume stored durably in Amazon S3.
- Only changed blocks since the last snapshot are saved, reducing storage cost and time.
- Used to create new volumes, migrate data across AZs/Regions, and as the basis for disaster recovery and AMI creation.
- Can be automated via Amazon Data Lifecycle Manager for scheduled backup/retention policies.

Q8. Compare Amazon S3 and Amazon EFS in terms of architecture and usage.

CO2 · 10M

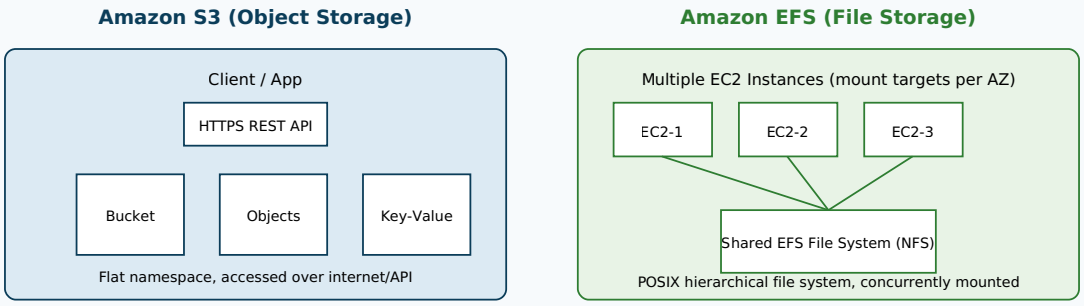


Fig 8.1 – S3 object storage vs EFS shared file storage architecture

Aspect	Amazon S3	Amazon EFS
Storage type	Object storage (flat, key-value)	File storage (hierarchical, POSIX)
Access	HTTPS REST API (PUT/GET)	NFS protocol, mounted on EC2
Concurrent access	Yes, but not a "file system"	Yes, natively shared across thousands of instances/ AZs
Use case	Backups, static websites, data lakes, media storage	Shared app data, content management, home directories
Scalability	Virtually unlimited, per-object	Elastic, auto-grows/shrinks with usage
Latency	Higher (API call per object)	Low latency (mounted like local disk)

Q9. Describe the lifecycle of an EC2 instance and outline the steps to manage snapshots. CO2 · 10M

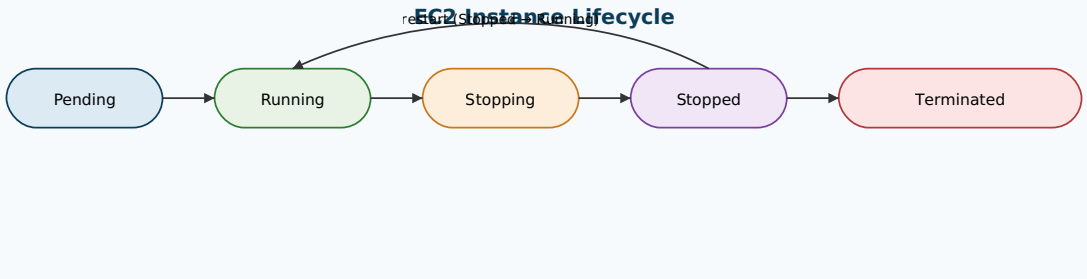


Fig 9.1 – EC2 instance state transitions

Lifecycle Stages

- **Launch (Pending):** AMI is selected, instance type/network/storage configured, instance boots.
- **Running:** instance is active and billed; application/services execute here.
- **Stopping/Stopped:** instance shut down (EBS-backed only); billing for compute stops, but EBS storage still billed; can be restarted later.
- **Rebooting:** OS-level restart without changing the underlying host in most cases.
- **Terminating/Terminated:** instance permanently deleted; root EBS volume deleted by default (unless "Delete on Termination" is disabled); this state is irreversible.

Managing Snapshots – Steps

- 1. Select the EBS volume attached to the instance in the EC2 console.
- 2. Choose *Actions → Create Snapshot*; add description/tags.
- 3. AWS stores the incremental snapshot in S3 (only changed blocks after the first full copy).
- 4. Use the snapshot to create a new volume, copy across Regions, or build a custom AMI.
- 5. Manage retention/automation using Amazon Data Lifecycle Manager (DLM) policies.
- 6. Delete obsolete snapshots to optimize storage cost.

Q10. How does Auto Scaling enhance application performance and reliability in AWS? CO2 · 10M

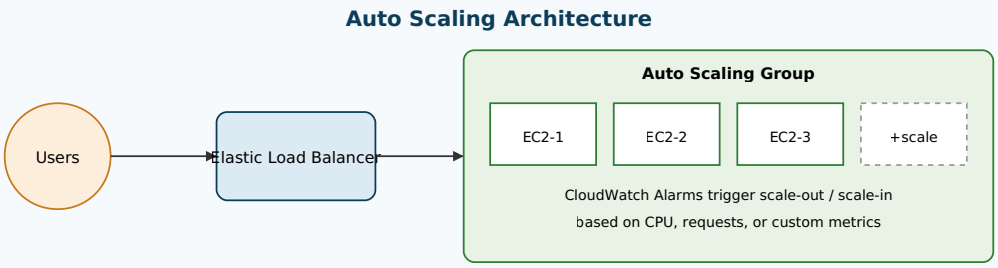


Fig 10.1 – ELB + Auto Scaling Group monitored by CloudWatch

- **Dynamic scaling:** automatically adds/removes EC2 instances based on CloudWatch metrics (CPU, network, custom app metrics), matching capacity to real-time demand.
- **Performance:** prevents overload during traffic spikes by launching new instances behind the load balancer, keeping response times consistent.
- **Reliability/Self-healing:** automatically replaces unhealthy or terminated instances to maintain the desired capacity.
- **Multi-AZ distribution:** spreads instances across AZs so a single AZ failure does not take the application down.
- **Cost efficiency:** scales in during low demand, avoiding paying for idle resources – realizing the elasticity characteristic of cloud computing.
- **Scaling policies:** Target Tracking, Step Scaling, Scheduled Scaling – allow fine control suited to predictable or unpredictable load patterns.

Q11. Explain the benefits of using AWS Local Zones for latency-sensitive applications. CO2 · 10M

What are AWS Local Zones?

- AWS Local Zones are infrastructure deployments that place select AWS services (compute, storage, database) closer to large population/industry centers, outside of a full Region, and connected back to the parent Region via AWS's private high-speed network.

Benefits for Latency-Sensitive Applications

- **Single-digit millisecond latency** for end users located near the Local Zone – critical for real-time applications.
- **Use cases:** real-time gaming, live video streaming/rendering, AR/VR, media & entertainment production, and real-time ad serving.
- **Local data processing:** allows compute-heavy tasks to run near the source of data generation, reducing round-trip time to a distant Region.
- **Same APIs/tools/AWS console** as the parent Region – no new operational skill set required.
- **Hybrid consistency:** integrates seamlessly with existing VPCs, enabling a consistent architecture between Region and Local Zone resources.
- **Data residency:** helps meet regulatory requirements where data must be processed within a specific city/state.

Example

- A media company rendering live sports graphics can deploy EC2/EBS in a Local Zone near the stadium's city, cutting latency compared to using the nearest full Region.

Q12. Describe the process of launching and connecting to an EC2 instance. Justify your selected instance type and connection method. CO2 · 10M

Steps to Launch an EC2 Instance

- 1. Sign in to the AWS Management Console → open EC2 → click *Launch Instance*.
- 2. Choose an Amazon Machine Image (AMI) – e.g., Amazon Linux 2023 or Ubuntu 22.04.
- 3. Select the instance type – e.g., **t3.medium** (2 vCPU, 4 GB RAM, burstable) for a small/medium web application – chosen for its cost-effective balance of compute and memory for variable workloads.
- 4. Configure network settings: VPC, subnet, and enable auto-assign public IP.
- 5. Add storage – attach a default gp3 EBS root volume.
- 6. Configure the Security Group – allow inbound SSH (port 22) from a specific IP, and HTTP/HTTPS (80/443) if hosting a web app.
- 7. Create/select a Key Pair (.pem file) – required for secure SSH authentication; download and store it safely.
- 8. Click *Launch Instance* and wait for the state to become "Running".

Connecting to the Instance

- **SSH (Linux):** `ssh -i mykey.pem ec2-user@<public-IP>` – chosen because it is the most secure, standard, low-overhead method for Linux instances using key-based authentication.
- **RDP (Windows):** using Remote Desktop Client with the decrypted administrator password.
- **EC2 Instance Connect / Session Manager:** browser-based, avoids managing/exposing SSH keys or opening port 22 to the internet – a good choice for tighter security posture.

Justification

- t3.medium balances cost and performance for general-purpose workloads; SSH via key pair is justified for its strong encryption and industry-standard practice for Linux server administration.

Q13. Demonstrate the deployment of a website using AWS Elastic Beanstalk and illustrate the architecture 'myapp.us-east-1.elasticbeanstalk.com'.

CO2 · 10M

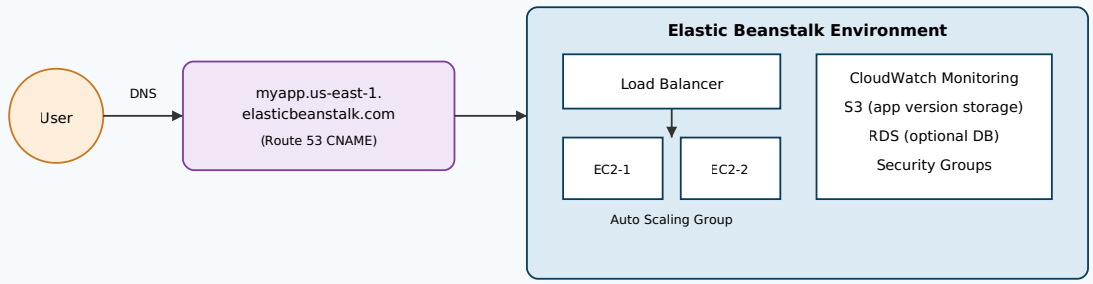


Fig 13.1 – Elastic Beanstalk environment accessed via its auto-generated domain

Deployment Steps

- 1. Open Elastic Beanstalk console → *Create Application* – name it (e.g., "myapp").
- 2. Select the platform (e.g., Node.js, Python, Java, or Docker).
- 3. Upload the application source bundle (zip of code) or use sample code.
- 4. Beanstalk automatically provisions: EC2 instances, an Auto Scaling Group, an Elastic Load Balancer, a Security Group, and CloudWatch alarms.
- 5. Beanstalk assigns a default URL in the form: **myapp.us-east-1.elasticbeanstalk.com** (application name + region + service domain).
- 6. Environment health, logs, and versions can be monitored/rolled back directly from the console.

Architecture Explanation

- Requests to the domain resolve via Route 53 to the Elastic Load Balancer, which distributes traffic across EC2 instances managed by an Auto Scaling Group – giving the developer a PaaS experience while AWS manages the underlying infrastructure.

Q18. Compare AWS Lambda and AWS Fargate architectures. Suggest a suitable scenario for each.

CO2 · 10M

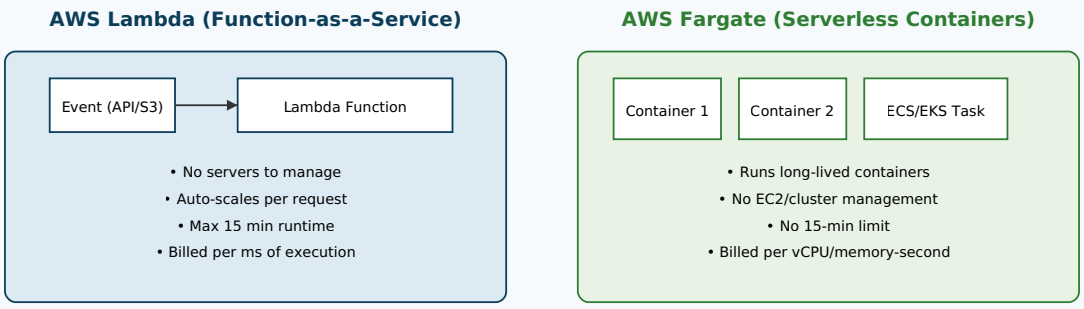


Fig 18.1 – Lambda (event-driven functions) vs Fargate (serverless containers)

Aspect	AWS Lambda	AWS Fargate
Compute unit	Individual function	Container (task/pod)
Execution model	Event-driven, short-lived (max 15 min)	Long-running processes, no fixed time limit
Cold start	Possible on infrequent invocation	Minimal once task is running
Scaling	Automatic, per-request, near-instant	Automatic via ECS/EKS service scaling
Billing	Per millisecond of execution + requests	Per vCPU/memory allocated per second
Best for	Lightweight, event-triggered tasks	Complex, stateful, long-running microservices

Suitable Scenarios

- **AWS Lambda:** processing S3 file-upload events, REST API backends (with API Gateway), scheduled cron-like jobs, real-time data transformation.
- **AWS Fargate:** running a full microservices application packaged in Docker containers, long-running batch jobs, or workloads requiring more than 15 minutes of execution and consistent CPU/memory.